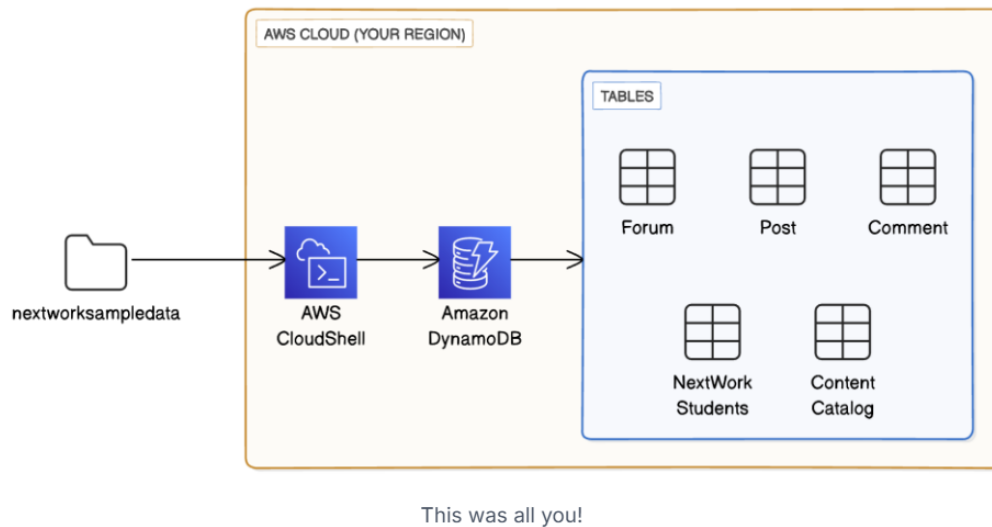


Load and Query DynamoDB Tables-2



Introducing Today's Project!

What is Amazon DynamoDB?

Amazon DynamoDB is a fully managed NoSQL database by AWS, offering fast, scalable, low-latency data storage. It supports key-value and document models, ideal for high-traffic apps, with built-in security, backup, and scalability features.

How I used Amazon DynamoDB in this project

In today's project, I used Amazon DynamoDB to create tables, add data, and run queries. I also set up a transaction to update two tables at once, ensuring data stays consistent. DynamoDB's speed and scalability made managing the data easy and efficient.

One thing I didn't expect in this project was...

One thing I didn't expect in this project was how important data modelling is for efficient querying in DynamoDB. I learned that queries must be carefully designed around the partition and sort keys to avoid errors and ensure fast data retrieval.

This project took me...

This project took me about 1hour to complete, including the time to set up the tables, load data, run queries, and work through the transaction steps. It was a great experience to see how DynamoDB handles scaling and consistency in real-time.

Querying DynamoDB Tables

A partition key is a unique identifier for each item in a DynamoDB table. It determines how data is distributed across partitions in the database. The partition key helps DynamoDB quickly locate and retrieve items based on its value.

A sort key is an additional attribute used to order items within a partition in DynamoDB. It allows multiple items with the same partition key but different sort key values, enabling efficient querying and retrieval of related data based on both keys.

The screenshot shows the AWS DynamoDB console interface. The 'Scan or query items' section is active, with the 'Query' option selected. The 'Select a table or index' dropdown is set to 'Table - Comment'. The 'Select attribute projection' dropdown is set to 'All attributes'. The 'Id (Partition key)' field contains the text 'I have a question/Just Complete Project #7 Dependencies and CodeArtifacts'. The 'CommentDateTime (Sort key)' field is set to 'Greater than' with a value of '2024-09-01'. The 'Sort descending' checkbox is unchecked. Below the query fields, there is a 'Filters' section with a 'Run' button and a 'Reset' button. A green status bar at the bottom of the query section indicates 'Completed. Read capacity units consumed: 0.5'. At the very bottom, it shows 'Items returned (1)' with buttons for 'Actions' and 'Create item'.

Limits of Using DynamoDB

I ran into an error when querying without using the Id (Partition key) because DynamoDB requires it for efficient data retrieval. This shows the importance of data modelling and how relational databases like SQL offer more flexible querying options.

Insights we could extract from the Comment table include comment counts per post, user engagement, and trending topics. Insights we can't easily extract include details about the original posts unless we join with the Posts table.

The screenshot shows the AWS DynamoDB console interface with an error. The 'Scan or query items' section is active, with the 'Query' option selected. The 'Select a table or index' dropdown is set to 'Table - Comment'. The 'Select attribute projection' dropdown is set to 'All attributes'. The 'Id (Partition key)' field is empty, and a red error message is displayed: 'Enter partition key value. The partition key filter cannot be empty.' The 'CommentDateTime (Sort key)' field is set to 'Greater than' with a value of 'Enter sort key value'. The 'Sort descending' checkbox is unchecked. Below the query fields, there is a 'Filters' section with a table showing a filter for 'PostId' with a value of 'User Abdulrahman'. At the bottom, it shows 'Items returned (1)' with buttons for 'Actions' and 'Create item'.

Running Queries with CLI

A query I ran in CloudShell was:

```
aws dynamodb get-item \  
  --table-name ContentCatalog \  
  --key '{"Id":{"N":"202"}}' \  
  --projection-expression "Title, ContentType, Services" \  
  --return-consumed-capacity TOTAL
```

This query will retrieve the Title, ContentType, and Services attributes of the item with Id 202 from the ContentCatalog table, along with details about the read capacity consumed.

Query options I could add to my query are:

1. --projection-expression: Limits the returned attributes to just the specified ones (e.g., Title, ContentType, Services).
2. --consistent-read: Ensures the most up-to-date data.
3. --return-consumed-capacity: Shows read capacity used.

```
~ $ aws dynamodb get-item \  
> --table-name ContentCatalog \  
> --key '{"Id":{"N":"202"}}' \  
> --projection-expression "Title, ContentType, Services" \  
> --return-consumed-capacity TOTAL \  
{  
  "Item": {  
    "Title": {  
      "S": "Don't miss out!"  
    },  
    "ContentType": {  
      "S": "Video"  
    }  
  },  
  "ConsumedCapacity": {  
    "TableName": "ContentCatalog",  
    "CapacityUnits": 0.5  
  }  
}  
~ $
```

Transactions

A transaction is a set of operations executed as a single unit, ensuring that all actions succeed or fail together. In DynamoDB, this means multiple operations are committed together, maintaining data consistency and integrity across tables.

I ran a transaction using the aws dynamodb transact-write-items command. This transaction did two things: it added a new comment to the Comment table and updated the Comments count in the Forum table by incrementing it, ensuring data consistency.

```
~ $ aws dynamodb transact-write-items --client-request-token TRANSACTION1 --transact-items '[  
> {  
>   "Put": {  
>     "TableName": "Comment",  
>     "Item": {  
>       "Id": {"S": "Events/Do a Project Together - NextWork Study Session"},  
>       "CommentDateTime": {"S": "2024-9-27T17:47:30Z"},  
>       "Comment": {"S": "Excited to attend!"},  
>       "PostedBy": {"S": "User Connor"}  
>     }  
>   },  
>   "Update": {  
>     "TableName": "Forum",  
>     "Key": {"Name": {"S": "Events"}},  
>     "UpdateExpression": "ADD Comments :inc",  
>     "ExpressionAttributeValues": {":inc": {"N": "1"} }  
>   }  
> }  
> ]'
```

Today you've learnt how to:

1. 🌟 **Create a DynamoDB table:** You could do this with both the AWS Management Console and AWS CLI.
2. ⬆️ **Load data into DynamoDB:** Using AWS CLI, you loaded four files into matching DynamoDB tables. Such a time saver!
3. 🔍 **Query data in the DynamoDB console:** This came with lots of learnings about partition and sort keys, and the benefits (and limits) of using DynamoDB compared to relational databases!